Dossier de Registro: padic-grn-search

Búsqueda y certificación de ordenamientos p-ádicos

Jesús Rogelio Pérez-Buendía Víctor Nopal-Coello Yuriria Cortés-Poza

2026-04-25

Abstract

`padic-grn-search` implementa estrategias de búsqueda para identificar y certificar ordenamientos relevantes en sistemas dinámicos discretos bajo el criterio de métricas p-ádicas. El dossier integra la justificación funcional del software, código ejecutable, pruebas automáticas y evidencia de corridas reproducibles para su registro formal.

Contents

1 Introducción **2**

2 Datos generales **2**
2.1 Correos electrónicos oficiales de autores 2
2.2 Financiamiento 2

3 Autoría oficial **2**

4 Documentación funcional completa **3**
4.1 README 3
4.2 Manual de usuario 4
4.3 Manual técnico 5
4.4 Configuración de paquete 6
4.5 Cita académica 6
4.6 Licencia 7
4.7 Configuración de pruebas 7

5 Código fuente completo **7**
5.1 Inicialización de paquete 7
5.2 CLI 8
5.3 Motor de búsqueda 9

6 Pruebas completas **12**
6.1 Prueba smoke 12
6.2 Pruebas de motor 12

7 Evidencia textual de validación **13**
7.1 Log de ejecución de pruebas 13
7.2 Salida GA 13
7.3 Salida exhaustiva min muE 13
7.4 Salida exhaustiva max muA 14

1 Introducción

Este software extiende el análisis del núcleo `padic-grn-core` mediante rutinas de optimización y exploración del espacio de permutaciones. Incluye métodos de búsqueda genética base y búsqueda exhaustiva para escenarios de tamaño controlado, priorizando verificabilidad de resultados y trazabilidad metodológica.

En términos de uso, `padic-grn-search` permite pasar de la evaluación de un ordering individual a la exploración sistemática de candidatos, facilitando estudios comparativos y procesos de validación en aplicaciones de redes de regulación genética y otros sistemas discretos.

2 Datos generales

- Nombre: `padic-grn-search`
- Versión: `0.1.0`
- Lenguaje: Python 3.10+
- Tipo: optimización y certificación de orderings p-ádicos
- DOI: pendiente de asignación institucional

2.1 Correos electrónicos oficiales de autores

- Jesús Rogelio Pérez-Buendía: `rogelio.perez@cimat.mx`
- Víctor Nopal-Coello: `victor_nopal@uaeh.edu.mx`
- Yuriria Cortés-Poza: `yuriria.cortes@iimas.unam.mx`

2.2 Financiamiento

- Modelos matemáticos y computacionales no convencionales para el estudio y análisis de problemas relevantes en Biología, CF 2019/217367.

3 Autoría oficial

```
# Autoría, ORCID y DOI oficial (softwares GRN)

Fecha: 2026-04-25

Este documento fija la información oficial de autoría para:

- `padic-grn-core`
- `padic-grn-search`
- `padic-grn-viz`

## Orden oficial de autoría

1. **Jesús Rogelio Pérez-Buendía**
   SECIHTI - Centro de Investigación en Matemáticas (CIMAT), Unidad Mérida (Mérida,
   ↪  Yucatán, México).
```

```
   ORCID: `0000-0002-7739-4779`

2. **Víctor Nopal-Coello**
   Universidad Autónoma del Estado de Hidalgo (UAEH) (Pachuca, Hidalgo, México).
   ORCID: `0000-0003-2608-3636`

3. **Yuriria Cortés-Poza**
   IIMAS, UNAM - Unidad Académica Yucatán (Mérida, Yucatán, México).
   ORCID: `0000-0003-1856-9206`

## Fuente de verificación (artículo y acuerdo interno)

- `20250115-p-adic-GRN-rewrite/articulo-final/parte-II.2/main-PII-2.tex`
  - autores y ORCID en bloque de autores
  - afiliaciones en bloque `\affil[...]`
- Acuerdo explícito de orden de autoría definido por el equipo para el paquete de
↪  software.

## DOI por software

- `padic-grn-core`: **pendiente de asignación** (Zenodo/release).
- `padic-grn-search`: **pendiente de asignación** (Zenodo/release).
- `padic-grn-viz`: **pendiente de asignación** (Zenodo/release).

Al asignarse, registrar aquí:

- DOI,
- versión/tag correspondiente,
- URL de depósito.

## Financiamiento (funding)

Modelos matemáticos y computacionales no convencionales para el estudio y análisis de
↪  problemas relevantes en Biología, CF 2019/217367.
```

4 Documentación funcional completa

4.1 README

```
# padic-grn-search: Búsqueda y certificación de orderings p-ádicos

## Resumen

Paquete de optimización y certificación de ordenamientos (`GA` + `B&B`) sobre el modelo
↪  discreto definido en `padic-grn-core`.

## Introducción

`padic-grn-search` amplía el uso del núcleo matemático para explorar múltiples orderings
↪  de manera sistemática. Permite comparar candidatos y certificar soluciones en
↪  escenarios donde la búsqueda exhaustiva o heurística es necesaria.

## Comandos
```

- `pgsearch-ga`
- `pgsearch-bb-min-muE`
- `pgsearch-bb-max-muA`

Nota operativa

La versión actual implementa:

- GA baseline nativo por muestreo aleatorio de permutaciones,
- búsqueda exhaustiva nativa (útil en `N` pequeño o con `--max-evaluations`).

Usa el núcleo matemático legado validado para el cálculo de métricas, manteniendo trazabilidad científica.

Autoría oficial

1. Jesús Rogelio Pérez-Buendía (SECIHTI-CIMAT, Unidad Mérida) - ORCID `0000-0002-7739-4779`
2. Víctor Nopal-Coello (Universidad Autónoma del Estado de Hidalgo, UAEH) - ORCID `0000-0003-2608-3636`
3. Yuriria Cortés-Poza (IIMAS, UNAM - Unidad Académica Yucatán) - ORCID `0000-0003-1856-9206`

DOI

- DOI del software: pendiente de asignacion institucional.

Financiamiento

Modelos matemáticos y computacionales no convencionales para el estudio y análisis de problemas relevantes en Biología, CF 2019/217367.

4.2 Manual de usuario

Manual de usuario - padic-grn-search

Instalación

```bash
cd software-suite/padic-grn-search
pip install -e .
```

Comandos principales

1) GA

```bash
pgsearch-ga ../padic-grn-core/examples/transitions_p2_n2_identity.csv 2 2 --objective minimize_mu_E --generations 5 --population 10
```

2) B&B mínimo de mu_E

```bash
```

```
pgsearch-bb-min-muE
```

3) B&B máximo de mu_A

```bash
pgsearch-bb-max-muA
```

Recomendaciones

- Para pruebas rápidas usar redes pequeñas (`N<=8`).
- Para corridas largas, usar scripts con checkpoint y monitorización.
- Registrar siempre parámetros y versión del software.

4.3 Manual técnico

```
# Manual técnico - padic-grn-search

## 1. Propósito

`padic-grn-search` resuelve optimización y certificación de ordenamientos:

- optimización heurística (GA),
- certificación exacta/semiexacta (B&B, según configuración).

## 2. Arquitectura actual

- `src/padic_grn_search/engine.py`
  - `random_ga_search(...)` (baseline nativo),
  - `exhaustive_search(...)` (nativo, truncable por `--max-evaluations`).
- `src/padic_grn_search/cli.py`
  - `pgsearch-ga`
  - `pgsearch-bb-min-muE`
  - `pgsearch-bb-max-muA`

La capa de búsqueda es nativa; el cálculo de métricas usa el core legado validado para
↪  conservar continuidad científica.

## 3. Diseño algorítmico (resumen)

- GA baseline nativo:
  - genera permutaciones aleatorias,
  - evalúa fitness por objetivo (`mu_E` o `-mu_A`),
  - conserva mejor solución global.
- Exhaustivo nativo:
  - recorre `N!` permutaciones,
  - opción de truncamiento por presupuesto de evaluaciones.

## 4. Entrada/Salida

Entrada:

- transiciones (`csv`),
- `N`, `p`,
```

```
- parámetros de corrida (`generations`, `population`, `seed`, `max-evaluations`).

Salida:

- mejor ordenamiento,
- métricas (`mu_E`, `mu_A`, `mu_I`),
- conteo de evaluaciones,
- indicador de truncamiento (si aplica).

## 5. Riesgos técnicos actuales

- dependencia del módulo core legado para evaluación de métricas,
- variabilidad por semillas si no se fija random seed,
- crecimiento factorial del exhaustivo para `N` grande.

Mitigación:

- migrar evaluación a API estable de `padic-grn-core`,
- definir protocolo de semillas en runbook,
- usar `--max-evaluations` y/o GA para escenarios grandes.
```

4.4 Configuración de paquete

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "padic-grn-search"
version = "0.1.0"
description = "Busqueda y certificacion de ordenamientos para sistemas discretos."
readme = "README.md"
requires-python = ">=3.10"
license = {text = "Proprietary - Pending institutional definition"}
authors = [
  {name = "Jesús Rogelio Pérez-Buendía", email = "rogelio.perez@cimat.mx"},
  {name = "Víctor Nopal-Coello", email = "victor_nopal@uaeh.edu.mx"},
  {name = "Yuriria Cortés-Poza", email = "yuriria.cortes@iimas.unam.mx"}
]
dependencies = []

[project.scripts]
pgsearch-ga = "padic_grn_search.cli:main_ga"
pgsearch-bb-min-muE = "padic_grn_search.cli:main_bb_min_mue"
pgsearch-bb-max-muA = "padic_grn_search.cli:main_bb_max_mua"

[tool.setuptools]
package-dir = {"" = "src"}

[tool.setuptools.packages.find]
where = ["src"]
```

4.5 Cita académica

```
cff-version: 1.2.0
```

```
message: "If you use this software, please cite it as below."
title: "padic-grn-search"
version: "0.1.0"
type: software
authors:
  - family-names: "Pérez-Buendía"
    given-names: "Jesús Rogelio"
    affiliation: "SECIHTI - Centro de Investigación en Matemáticas (CIMAT), Unidad
    ↪  Mérida"
    orcid: "https://orcid.org/0000-0002-7739-4779"
  - family-names: "Nopal-Coello"
    given-names: "Víctor"
    affiliation: "Universidad Autónoma del Estado de Hidalgo (UAEH)"
    orcid: "https://orcid.org/0000-0003-2608-3636"
  - family-names: "Cortés-Poza"
    given-names: "Yuriria"
    affiliation: "Instituto de Investigaciones en Matemáticas Aplicadas y en Sistemas
    ↪  (IIMAS), UNAM -- Unidad Académica Yucatán"
    orcid: "https://orcid.org/0000-0003-1856-9206"
abstract: "Módulo de búsqueda y certificación de ordenamientos óptimos mediante
↪  algoritmos genéticos y branch-and-bound para sistemas dinámicos discretos."
license: "Proprietary - Pending institutional definition"
repository-code: "PENDING_REPOSITORY_URL"
doi: "10.0000/pending-doi"
```

4.6 Licencia

```
Proprietary License Notice
Copyright (c) 2026
Jesús Rogelio Pérez-Buendía, Víctor Nopal-Coello, Yuriria Cortés-Poza

This software is currently released for academic evaluation and registration
workflow preparation. Redistribution, sublicensing, and commercial use are not
authorized without explicit written permission from the rights holders and
institutional approval.

Final license terms may be updated in the public release.
```

4.7 Configuración de pruebas

```
[pytest]
pythonpath = src
```

5 Código fuente completo

5.1 Inicialización de paquete

```
"""padic-grn-search package."""

__all__ = ["__version__"]
__version__ = "0.1.0"
```

5.2 CLI

```
"""CLI funcional nativa para padic-grn-search."""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

from .engine import exhaustive_search, random_ga_search


def _write_json(path: str, payload: dict) -> Path:
    out = Path(path).resolve()
    out.parent.mkdir(parents=True, exist_ok=True)
    with out.open("w", encoding="utf-8") as f:
        json.dump(payload, f, indent=2, ensure_ascii=False)
    return out


def main_ga() -> None:
    parser = argparse.ArgumentParser(description="Búsqueda GA nativa (baseline) para
    ↪  ordenamientos.")
    parser.add_argument("transitions_csv")
    parser.add_argument("N", type=int)
    parser.add_argument("p", type=int)
    parser.add_argument("--objective", choices=["minimize_mu_E", "maximize_mu_A"],
    ↪  default="minimize_mu_E")
    parser.add_argument("--generations", type=int, default=50)
    parser.add_argument("--population", type=int, default=30)
    parser.add_argument("--seed", type=int, default=None)
    parser.add_argument("--output-json", default="search_ga_result.json")
    args = parser.parse_args()
    try:
        result = random_ga_search(
            transitions_csv=args.transitions_csv,
            n=args.N,
            p=args.p,
            objective=args.objective,
            generations=args.generations,
            population=args.population,
            seed=args.seed,
        )
        out = _write_json(args.output_json, result)
        print(f"OK: resultado GA guardado en {out}")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgsearch-ga: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc


def main_bb_min_mue() -> None:
    parser = argparse.ArgumentParser(description="Búsqueda exhaustiva nativa para
    ↪  minimizar mu_E.")
```

```
    parser.add_argument("transitions_csv")
    parser.add_argument("N", type=int)
    parser.add_argument("p", type=int)
    parser.add_argument("--max-evaluations", type=int, default=None)
    parser.add_argument("--output-json", default="search_bb_min_result.json")
    args = parser.parse_args()
    try:
        result = exhaustive_search(
            transitions_csv=args.transitions_csv,
            n=args.N,
            p=args.p,
            objective="minimize_mu_E",
            max_evaluations=args.max_evaluations,
        )
        out = _write_json(args.output_json, result)
        print(f"OK: resultado BB-min guardado en {out}")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgsearch-bb-min-muE: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc


def main_bb_max_mua() -> None:
    parser = argparse.ArgumentParser(description="Búsqueda exhaustiva nativa para
    ↪  maximizar mu_A.")
    parser.add_argument("transitions_csv")
    parser.add_argument("N", type=int)
    parser.add_argument("p", type=int)
    parser.add_argument("--max-evaluations", type=int, default=None)
    parser.add_argument("--output-json", default="search_bb_max_result.json")
    args = parser.parse_args()
    try:
        result = exhaustive_search(
            transitions_csv=args.transitions_csv,
            n=args.N,
            p=args.p,
            objective="maximize_mu_A",
            max_evaluations=args.max_evaluations,
        )
        out = _write_json(args.output_json, result)
        print(f"OK: resultado BB-max guardado en {out}")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgsearch-bb-max-muA: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc
```

5.3 Motor de búsqueda

```
"""Motor nativo inicial para búsqueda en padic-grn-search."""

from __future__ import annotations

import importlib.util
import itertools
import random
from pathlib import Path
from typing import Any
```

```
def workspace_root() -> Path:
    current = Path(__file__).resolve()
    for parent in current.parents:
        if (parent / "20250115-p-adic-GRN-rewrite").exists():
            return parent
    raise RuntimeError("No se encontró la raíz del workspace.")


def load_legacy_core_module() -> Any:
    root = workspace_root()
    module_path = (
        root
        / "20250115-p-adic-GRN-rewrite"
        / "articulo-final"
        / "supplementary"
        / "code"
        / "core"
        / "compute_mu_generalized.py"
    )
    if not module_path.exists():
        raise FileNotFoundError(f"No existe módulo core legado: {module_path}")
    spec = importlib.util.spec_from_file_location("legacy_core_module", module_path)
    if spec is None or spec.loader is None:
        raise RuntimeError("No se pudo crear spec del módulo core legado.")
    mod = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(mod)
    return mod


def evaluate_ordering(
    transitions_csv: str,
    n: int,
    p: int,
    ordering: list[int],
) -> dict[str, int]:
    core = load_legacy_core_module()
    table = core.read_transition_table_csv(transitions_csv, p)
    scores = core.compute_scores(p, n, ordering, table, return_profiles=False)
    return {"mu_E": scores["mu_E"], "mu_A": scores["mu_A"], "mu_I": scores["mu_I"]}


def random_ga_search(
    transitions_csv: str,
    n: int,
    p: int,
    objective: str = "minimize_mu_E",
    generations: int = 50,
    population: int = 30,
    seed: int | None = None,
) -> dict[str, Any]:
    rng = random.Random(seed)
    all_idx = list(range(n))
    best_order = None
```

```
    best_fit = None
    best_scores = None

    core = load_legacy_core_module()
    table = core.read_transition_table_csv(transitions_csv, p)
    evals = 0
    for _ in range(generations):
        for _ in range(population):
            cand = all_idx[:]
            rng.shuffle(cand)
            scores = core.compute_scores(p, n, cand, table, return_profiles=False)
            fit = scores["mu_E"] if objective == "minimize_mu_E" else -scores["mu_A"]
            evals += 1
            if best_fit is None or fit < best_fit:
                best_fit = fit
                best_order = cand[:]
                best_scores = {
                    "mu_E": scores["mu_E"],
                    "mu_A": scores["mu_A"],
                    "mu_I": scores["mu_I"],
                }

    return {
        "method": "native_random_ga_baseline",
        "objective": objective,
        "evaluations": evals,
        "best_ordering": best_order,
        "best_fitness": best_fit,
        "best_scores": best_scores,
    }


def exhaustive_search(
    transitions_csv: str,
    n: int,
    p: int,
    objective: str = "minimize_mu_E",
    max_evaluations: int | None = None,
) -> dict[str, Any]:
    core = load_legacy_core_module()
    table = core.read_transition_table_csv(transitions_csv, p)

    best_order = None
    best_fit = None
    best_scores = None
    evals = 0

    for perm in itertools.permutations(range(n)):
        if max_evaluations is not None and evals >= max_evaluations:
            break
        scores = core.compute_scores(p, n, list(perm), table, return_profiles=False)
        fit = scores["mu_E"] if objective == "minimize_mu_E" else -scores["mu_A"]
        evals += 1
        if best_fit is None or fit < best_fit:
            best_fit = fit
```

```
            best_order = list(perm)
            best_scores = {
                "mu_E": scores["mu_E"],
                "mu_A": scores["mu_A"],
                "mu_I": scores["mu_I"],
            }

    return {
        "method": "native_exhaustive",
        "objective": objective,
        "evaluations": evals,
        "best_ordering": best_order,
        "best_fitness": best_fit,
        "best_scores": best_scores,
        "truncated": max_evaluations is not None and evals >= max_evaluations,
    }
```

6 Pruebas completas

6.1 Prueba smoke

```
from padic_grn_search import __version__


def test_version_defined() -> None:
    assert isinstance(__version__, str)
    assert __version__ == "0.1.0"
```

6.2 Pruebas de motor

```
from __future__ import annotations

from pathlib import Path

from padic_grn_search.engine import exhaustive_search, random_ga_search


def _write_identity_csv(path: Path, p: int, n: int) -> None:
    total = p**n
    rows = ["config_from,config_to"]
    rows.extend(f"{i},{i}" for i in range(total))
    path.write_text("\n".join(rows), encoding="utf-8")


def test_random_ga_search_returns_result(tmp_path: Path) -> None:
    csv_path = tmp_path / "transitions.csv"
    _write_identity_csv(csv_path, p=2, n=2)
    out = random_ga_search(
        transitions_csv=str(csv_path),
        n=2,
        p=2,
        generations=2,
        population=4,
        seed=7,
```

```
    )
    assert out["best_ordering"] in ([0, 1], [1, 0])
    assert set(out["best_scores"].keys()) == {"mu_E", "mu_A", "mu_I"}


def test_exhaustive_search_truncated(tmp_path: Path) -> None:
    csv_path = tmp_path / "transitions.csv"
    _write_identity_csv(csv_path, p=2, n=3)
    out = exhaustive_search(
        transitions_csv=str(csv_path),
        n=3,
        p=2,
        max_evaluations=2,
    )
    assert out["evaluations"] == 2
    assert out["truncated"] is True
```

7 Evidencia textual de validación

7.1 Log de ejecución de pruebas

```
...                                                                      [100%]
3 passed in 0.02s
```

7.2 Salida GA


```
{
  "method": "native_random_ga_baseline",
  "objective": "minimize_mu_E",
  "evaluations": 12,
  "best_ordering": [
    0,
    1
  ],
  "best_fitness": 0,
  "best_scores": {
    "mu_E": 0,
    "mu_A": 0,
    "mu_I": 4
  }
}
```


7.3 Salida exhaustiva min muE


```
{
  "method": "native_exhaustive",
  "objective": "minimize_mu_E",
  "evaluations": 2,
  "best_ordering": [
    0,
    1
  ],
  "best_fitness": 0,
  "best_scores": {
```

```
    "mu_E": 0,
    "mu_A": 0,
    "mu_I": 4
  },
  "truncated": true
}
```


7.4 Salida exhaustiva max muA


```
{
  "method": "native_exhaustive",
  "objective": "maximize_mu_A",
  "evaluations": 2,
  "best_ordering": [
    0,
    1
  ],
  "best_fitness": 0,
  "best_scores": {
    "mu_E": 0,
    "mu_A": 0,
    "mu_I": 4
  },
  "truncated": true
}
```